SPEAKMATE

Feature Specification

STORY CREATION FLOW

Chat-based Story Building

Luồng tạo câu chuyện qua chat với Mentor Ni

Phiên bản:	1.0
Ngày:	21/04/2026
Đội ngũ:	NEXT GENTS
Phụ thuộc:	Story Bank Spec v1.0
Trạng thái:	Draft

1. Tổng quan

Vấn đề cần giải quyết

Phiên bản hiện tại cố định hỏi 2 câu bổ sung bất kể chất lượng input. Nếu user đưa quá ít thông tin, hệ thống vẫn chấp nhận và sinh story yếu. Nếu user đưa đủ thông tin, vẫn bị bắt trả lời 2 câu thừa.

Giải pháp mới

Chuyển thẳng vào màn hình chat với Mentor Ni. Ni hỏi thêm để làm giàu câu chuyện, nhưng user LUÔN có quyền bấm "Cấu trúc hóa" bất kỳ lúc nào. Ni không gác cổng, không chặn, không đánh giá đủ/thiếu. Ni chỉ khai thác thêm.

1.1. Nguyên tắc thiết kế

- **User có quyền quyết định:** nút Cấu trúc hóa luôn hiện, luôn bấm được, không bị disable.
- **Ni chỉ khai thác:** hỏi thêm để làm giàu, không đánh giá đủ hay thiếu, không nói "cần thêm thông tin".
- **Chấp nhận thiếu:** nếu user bấm cấu trúc hóa sớm, hệ thống vẫn tạo story, đánh dấu field thiếu nhẹ nhàng.
- **Không giới hạn:** không cố định số lượt hỏi. User muốn chat 1 lượt hay 15 lượt đều ok.

2. User flow

2.1. Bước 1: Màn hình khởi tạo (/stories/create)

- User chọn framework: STAR / PREP / CAR
- User nhập ý tưởng ban đầu (text area hoặc voice input)
- Bấm "Khởi tạo câu chuyện"
- Chuyển thẳng sang màn hình chat

2.2. Bước 2: Màn hình chat với Mentor Ni

Ni nhận input gốc, phản hồi tích cực, rồi hỏi câu đầu tiên nhắm vào field còn thiếu của framework đã chọn.

> **Ví dụ luồng chat**
>
> Input gốc: "em làm backend đồ án CNPM, bị lỗi DB lúc demo" Ni: "Hay đó! Đồ án nhóm mấy người vậy?" User: "4 người" Ni: "Lúc bị lỗi DB, bạn cụ thể đã làm gì để sửa?" User: "check log, thấy connection string sai, sửa trong 10p" Ni: "Kết quả cuối cùng thế nào?" User: "nhóm được 9 điểm" Ni: "Tuyệt! Mình đủ thông tin rồi. Bấm Cấu trúc hóa khi bạn sẵn sàng nhé!" (User có thể bấm Cấu trúc hóa bất kỳ lúc nào trong quá trình này)

2.3. Bước 3: Cấu trúc hóa

User bấm nút "Cấu trúc hóa câu chuyện". Hệ thống thu thập toàn bộ context (input gốc + lịch sử chat) và gọi Gemini 2.0 Flash để sinh story có cấu trúc.

- Output: structured content theo framework, fullScript 30-60 giây, tags gợi ý, thời lượng ước tính
- Nếu thiếu field: vẫn tạo story, đánh dấu missingFields, hiện note nhẹ nhàng
- User KHÔNG bị bắt quay lại bổ sung

2.4. Bước 4: Preview và lưu

- Hiện preview story đã cấu trúc theo framework
- Hiện fullScript với nút "Nghe thử" (TTS)
- Tags gợi ý (chỉnh sửa được)
- Nếu có missingFields: hiện note + nút "Quay lại chat bổ sung"
- Nút "Lưu vào Story Bank" và "Chỉnh sửa"

3. Luồng kỹ thuật

3.1. Khởi tạo chat session

1. User bấm "Khởi tạo" → frontend tạo StoryCreationSession, lưu initialInput + framework
2. Gửi lên backend: { framework, rawInput }
3. Backend gọi Mentor Ni (Gemma 4 local) với prompt khai thác
4. Ni trả về: chatMessage (phản hồi tích cực + câu hỏi đầu tiên) + fieldTargeted
5. Frontend hiện chat bubble của Ni + nút Cấu trúc hóa ở dưới

3.2. Vòng lặp chat

1. User trả lời → frontend gửi lên backend kèm toàn bộ context tích lũy
2. Backend gọi Ni với: initialInput + tất cả lượt chat trước + câu trả lời mới
3. Ni trả về: chatMessage + fieldTargeted
4. Frontend hiện bubble mới. Lặp lại cho đến khi user bấm Cấu trúc hóa.

3.3. Cấu trúc hóa

1. User bấm nút → frontend thu thập: initialInput + toàn bộ chat messages
2. Gửi lên backend → gọi Gemini 2.0 Flash (The Brain) với prompt cấu trúc hóa
3. Validation Gate (Zod) kiểm tra output đúng schema
4. Trả về frontend: structured content, fullScript, tags, missingFields
5. Hiện màn hình preview

3.4. Lưu story

1. User review preview, chỉnh sửa nếu cần
2. Bấm Lưu → tạo StoryEntry trong Supabase
3. Xóa StoryCreationSession (dữ liệu tạm)
4. Chuyển về Story Bank với story mới đã hiện

4. Cấu trúc dữ liệu

4.1. StoryCreationSession (dữ liệu tạm trong quá trình tạo)

Field	Type	Mô tả
`id`	string	UUID
`userId`	string	FK tới users
`framework`	enum	'STAR' | 'PREP' | 'CAR'
`initialInput`	string	Ý tưởng ban đầu user nhập
`inputMethod`	enum	'text' | 'voice'
`chatMessages`	JSON[]	Mảng { role, content, fieldTargeted } của từng lượt chat
`totalTurns`	number	Số lượt chat đã thực hiện
`status`	enum	'chatting' | 'structuring' | 'previewing' | 'saved'
`createdAt`	timestamp	Thời điểm bắt đầu tạo

4.2. ChatMessage (trong chatMessages array)

Field	Type	Mô tả
`role`	enum	'user' | 'mentor'
`content`	string	Nội dung tin nhắn
`fieldTargeted`	string | null	Field mà Ni đang hỏi về (situation/task/action/result). Null cho tin của user.
`timestamp`	Date	Thời điểm gửi

4.3. StructuredOutput (output của Gemini khi cấu trúc hóa)

Field	Type	Mô tả
`title`	string	Tiêu đề ngắn: "Fix lỗi DB lúc demo"
`structured`	JSON	{ situation, task, action, result } (hoặc tương đương theo framework)
`fullScript`	string	Đoạn nói hoàn chỉnh 30-60 giây, ngôn ngữ tự nhiên
`estimatedDuration`	number	Thời lượng nói ước tính (giây)
`suggestedTags`	string[]	Tags gợi ý dựa trên nội dung

`missingFields`	string[]	Fields thiếu thông tin: ["result"] hoặc []
`completenessNote`	string | null	Ghi chú về field thiếu nếu có, VD: "Phần Result chưa rõ — bổ sung sau?"

5. Thiết kế prompt

5.1. Prompt cho Mentor Ni (khai thác ý tưởng)

System prompt

Bạn là Ni, đang giúp user khai thác ý tưởng để xây dựng câu chuyện theo framework {framework}. User đã nhập: {raw_input}. Quy tắc: (1) Mỗi lượt CHỈ hỏi MỘT câu, ngắn gọn, cụ thể. (2) Hỏi theo thứ tự ưu tiên field còn thiếu của {framework}. (3) Không hỏi lại thông tin user đã cung cấp. (4) Nếu user trả lời ngắn, chấp nhận rồi hỏi field tiếp — không ép kể dài. (5) Tone: tò mò, bạn bè. "Rồi sao nữa?", "Kết quả thế nào?". (6) KHÔNG BAO GIỜ nói "chưa đủ thông tin" hay "cần thêm". User tự quyết định khi nào cấu trúc hóa. Trả về JSON: { chatMessage: string, fieldTargeted: string }

5.2. Prompt cấu trúc hóa (Gemini 2.0 Flash)

System prompt khi user bấm Cấu trúc hóa

Cấu trúc hóa câu chuyện theo framework {framework} từ cuộc trò chuyện sau. Input gốc: {raw_input}. Lịch sử chat: {chat_history}. Quy tắc: (1) Cấu trúc theo {framework}. (2) Viết fullScript tự nhiên 30-60 giây. (3) Nếu thiếu field, VẪN cấu trúc hóa nhưng đánh dấu vào missingFields. (4) Gợi ý tags. (5) Ước tính thời lượng nói. Trả về JSON theo StructuredOutput schema.

5.3. Cách Ni chọn field để hỏi

Ni hỏi theo thứ tự ưu tiên sau (cho STAR). Field nào đã có thông tin thì bỏ qua:

Ưu tiên	Field	Vì sao	Câu hỏi mẫu
1	Action	Quan trọng nhất: user làm gì	"Lúc đó bạn cụ thể đã làm gì?"
2	Result	Thuyết phục nhất khi kể	"Kết quả cuối cùng thế nào?"
3	Situation	Bối cảnh giúp người nghe hiểu	"Đồ án này có mấy người?"
4	Task	Vai trò của user	"Phần nào là do bạn phụ trách?"

Lý do Action và Result ưu tiên hơn: đây là hai phần mà user thường bỏ quên nhất khi kể chuyện, và cũng là hai phần thuyết phục nhất với người nghe. Situation và Task thường đã có sẵn trong input ban đầu.

6. Hướng dẫn thiết kế UI

6.1. Màn hình khởi tạo (/stories/create)

- Dropdown chọn framework (STAR mặc định) với tooltip giải thích ngắn mỗi framework
- Text area lớn để nhập ý tưởng + nút voice input
- Placeholder gợi ý: "Kể ngắn về một trải nghiệm bạn tự hào — ví dụ: đồ án, cuộc thi, lần giải quyết vấn đề..."
- Nút "Khởi tạo câu chuyện" ở dưới

6.2. Màn hình chat tạo story

- Header: "Tạo câu chuyện mới" + badge framework (STAR/PREP/CAR) + nút quay lại
- Body: chat bubbles (user bên phải, Ni bên trái)
- Tin đầu tiên của Ni: phản hồi tích cực về input gốc + câu hỏi đầu tiên
- Footer: input text + nút Gửi
- Nút "Cấu trúc hóa câu chuyện" CỐ ĐỊNH Ở DƯỚI CÙNG, không bị cuộn, luôn hiện, luôn bấm được
- Nút này nổi bật (primary color, size lớn) để user biết luôn có thể bấm

6.3. Màn hình preview

- Hiện structured content theo framework (mỗi field là một section)
- fullScript trong khung riêng với nút "Nghe thử"
- Tags dạng chips, chỉnh sửa được (thêm/xóa)
- Thời lượng ước tính: "~40 giây"
- Nếu missingFields không rỗng: banner nhẹ nhàng màu vàng: "Phần Result chưa rõ — bạn có thể bổ sung sau" + nút "Quay lại chat"
- 2 nút chính: "Lưu vào Story Bank" và "Chỉnh sửa thủ công"

7. Edge cases

Tình huống	Xử lý
User bấm Cấu trúc hóa ngay sau khi khởi tạo (0 lượt chat)	Vẫn cấu trúc hóa từ initialInput. Sẽ có nhiều missingFields nhưng không chặn.
User nhập input rất đầy đủ ngay từ đầu	Ni vẫn hỏi 1 câu mở rộng (VD: cảm xúc, bài học rút ra). User bấm Cấu trúc hóa luôn cũng được.
User trả lời rất ngắn ("ok", "9 điểm")	Ni chấp nhận, chuyển sang field tiếp theo. Không ép kể dài.
User muốn kể chuyện không liên quan	Ni lịch sự dẫn về: "Hay đó! Nhưng để mình tập trung vào câu chuyện này trước nhé."
User quay lại chat từ preview để bổ sung	Restore toàn bộ chat history. Ni tiếp tục hỏi từ chỗ dừng. Bấm Cấu trúc hóa lại → gọi Gemini lần 2 với context mới.
User đóng app giữa chừng	StoryCreationSession lưu trạng thái 'chatting'. Khi mở lại, hiện popup: "Bạn có câu chuyện đang tạo dở — tiếp tục?"
Gemini trả về sai schema khi cấu trúc hóa	Validation Gate từ chối. Retry tối đa 2 lần. Nếu vẫn fail: hiện lỗi nhẹ nhàng, cho user thử lại hoặc quay lại chat.

8. Acceptance criteria

#	Tiêu chí	Ưu tiên	Trạng thái
1	Chọn framework + nhập input + bấm Khởi tạo → vào chat	P0	Chưa làm
2	Ni hỏi thêm từng câu một, nhắm vào field còn thiếu	P0	Chưa làm
3	Nút Cấu trúc hóa luôn hiện, luôn bấm được, không bị disable	P0	Chưa làm
4	Bấm Cấu trúc hóa → Gemini sinh story JSON → Validation Gate	P0	Chưa làm
5	Preview hiện structured + fullScript + tags + missingFields	P0	Chưa làm
6	Lưu story thành công vào Supabase	P0	Chưa làm
7	Voice input trên màn hình khởi tạo (PhoWhisper)	P1	Chưa làm
8	Quay lại chat từ preview để bổ sung + cấu trúc hóa lại	P1	Chưa làm
9	Lưu session dở khi đóng app giữa chừng	P1	Chưa làm
10	Nút "Nghe thử" TTS trên preview	P2	Chưa làm
11	Hỗ trợ framework PREP và CAR ngoài STAR	P2	Chưa làm

9. Lưu ý cho dev

> **Nguyên tắc vàng**
> User LUÔN có quyền cấu trúc hóa. Không gate, không chặn, không disable nút. Nếu thiếu thông tin thì story sẽ yếu hơn — nhưng đó là quyền của user. Hệ thống chỉ thông báo, không cấm.

9.1. Prompt safety

- Ni KHÔNG BAO GIỜ nói: "chưa đủ", "cần thêm", "không thể tạo"
- Ni KHÔNG đánh giá chất lượng input của user
- Ni CHỈ hỏi thêm và chấp nhận câu trả lời
- Nếu user hỏi "thế này đủ chưa?" → Ni trả lời: "Bạn có thể bấm Cấu trúc hóa bất kỳ lúc nào, hoặc kể thêm cho câu chuyện đầy đặn hơn"

9.2. Context management

- Mỗi lượt gọi Ni: gửi initialInput + TOÀN BỘ chat history
- Khi cấu trúc hóa: gửi initialInput + TOÀN BỘ chat history cho Gemini
- Không tóm tắt hay lược bỏ — mọi chi tiết user nói đều có giá trị

9.3. Performance

- Ni trả lời mỗi lượt: target < 1.5 giây (Gemma 4 local)
- Cấu trúc hóa: target < 5 giây (Gemini 2.0 Flash API + Validation)
- Preview render: instant (dữ liệu đã có sẵn từ API response)

9.4. Privacy

- StoryCreationSession là dữ liệu tạm — xóa sau khi user lưu hoặc hủy story
- Chat trong quá trình tạo KHÔNG lưu vĩnh viễn — chỉ giữ trong session
- Chỉ StoryEntry cuối cùng được lưu vào Supabase